# Relatório de Ajustes – Space Invaders Lateral

Projeto de Programação Orientada a Objetos · 15 de outubro de 2026

## 1. Objetivo

Este relatório descreve três melhorias feitas no jogo *Space Invaders Lateral* (HTML, CSS e JavaScript, com classes), explicando o que mudou, em quais arquivos e por que cada decisão foi tomada dessa forma.

1. A nave do jogador deve **sinalizar visualmente quando recebe dano** (piscar e ficar vermelha).
2. Os tiros do jogador **não devem mais destruir os escudos** (blocos azuis à esquerda da tela).
3. Os escudos devem ficar **um pouco mais à frente**, mais afastados da nave.

## 2. Resumo das alterações

| Arquivo | O que foi feito |
|---|---|
| `js/Jogador.js` | Novo atributo `tempoDano`, novo método `receberDano()`, e `desenhar()` e `atualizar()` ajustados. |
| `js/Jogo.js` | Dois pontos de dano passaram a chamar `receberDano()`, e a colisão com os escudos passou a considerar só tiros inimigos. Os escudos também foram deslocados 80 px para a frente (constante X_ESCUDOS). |
| `js/Barreira.js` | Apenas um comentário atualizado (sem mudança de comportamento). |
| `antes/ajuste_dano_escudo/` | Cópias dos arquivos originais, para comparação antes/depois. |

# 3. Alteração 1: nave pisca em vermelho ao levar dano

## Como era

Quando a nave era atingida, o `Jogo` simplesmente fazia `this.jogador.vidas--`. A única indicação era o contador de vidas no HUD e um som.

## Como ficou

A classe `Jogador` ganhou a responsabilidade de lidar com o próprio dano:

- **`tempoDano`**: contador regressivo em frames (1 frame = 1/60 s). Valor 0 significa nave normal; maior que 0 significa nave em efeito de dano.
- **`receberDano()`**: tira uma vida **e** inicia o efeito (`tempoDano = DURACAO_DANO`). Se a nave for atingida de novo durante o efeito, ele recomeça.
- **`desenhar()`**: enquanto `tempoDano > 0`, alterna a cor entre vermelho (`#ff2a2a`) e o verde normal a cada 6 frames. A cor é escolhida numa variável local, sem alterar `this.cor`, para que a cor original nunca se perca. O cálculo usa o tempo *decorrido* desde o dano, então o efeito começa em vermelho e termina na cor normal.
- **`atualizar()`**: além de contar o cooldown do tiro, diminui `tempoDano` a cada frame. Como só roda dentro do loop do jogo, o efeito congela junto com a pausa.
- **Constantes estáticas** (`Jogador.DURACAO_DANO = 48`, `INTERVALO_PISCADA = 6`, `COR_DANO`): evitam números soltos no código e facilitam ajustes, seguindo o mesmo padrão de `Barreira.TAMANHO`.

No `Jogo.js`, os dois locais onde a nave é atingida (choque com invasor e tiro inimigo) passaram a chamar `this.jogador.receberDano()` no lugar de `vidas--`.

## Por que dessa forma

O `Jogo` apenas **avisa** que a nave foi atingida, e o `Jogador` **decide** o que acontece. Assim, a regra de dano fica em um único lugar. Se o grupo quiser acrescentar depois uma invencibilidade temporária ou outro efeito, basta mexer em `receberDano()`.

# 4. Alteração 2: tiros do jogador não destroem os escudos

## Como era

Em `_verificarColisoes()`, o teste de colisão com as barreiras percorria os tiros do jogador **e** dos inimigos:

```
[...this.projeteisJogador, ...this.projeteisInimigo].forEach((projetil) => {
... });
```

### Como ficou

```
this.projeteisInimigo.forEach((projetil) => { ... });
```

Os tiros do jogador deixaram de entrar nesse teste. Por isso, eles **atravessam** o escudo sem danificá-lo. Os tiros inimigos continuam perdendo resistência dos blocos normalmente, e a colisão dos tiros do jogador com os invasores não foi alterada.

### Por que dessa forma

Foi a menor mudança possível: não exigiu atributos novos nem condições extras, apenas restringir a lista de tiros verificada. O escudo passa a cumprir só o papel de proteger o jogador.

### Decisão de projeto

O enunciado permitia duas interpretações: o tiro **atravessar** o escudo ou ser **barrado** por ele sem causar dano. Foi escolhida a primeira, porque os escudos ficam na frente da nave. Se o tiro fosse barrado, o jogador não conseguiria acertar os invasores sempre que estivesse alinhado com um escudo. A outra opção é simples de implementar, caso o grupo prefira.

## 5. Alteração 3: escudos mais à frente

Os escudos foram deslocados 80 px para a direita (de x = 130 para x = 210), ficando mais afastados da nave.

### Como ficou

Em `_criarBarreiras()` (`Jogo.js`), a posição horizontal era um número solto (`const x = 130`). Agora ela vem da constante `Jogo.X_ESCUDOS = 210`, declarada junto das demais constantes do jogo. Para afastar ou aproximar os escudos, basta mudar esse único valor.

### Por que dessa forma

Uma constante nomeada deixa claro o que o número significa e evita procurar o valor dentro do método. O deslocamento de 80 px foi escolhido para ser perceptível sem ser exagerado.

### Verificação

Os escudos ocupam de x = 210 a 258 e a nave nasce em x = 40 a 70, sem encostar neles. Na menor largura de tela do jogo (960 px), a coluna de invasores mais próxima da nave (horda 4) começa em x = 486, então ainda sobra espaço. Como o caminho dos invasores até os escudos ficou 80 px mais curto, eles chegam a eles um pouco mais cedo.

## 6. Como escudos, jogador e horda aparecem na tela

O jogo separa duas etapas: **criar** os objetos (posição e dados) e **desenhar** (pintar no canvas). A classe `Jogo` coordena as duas, e cada objeto sabe se desenhar.

| Elemento | Onde é criado | Onde é desenhado |
|---|---|---|
| Jogador | `_resetarEstado()`: `new Jogador(40, 0)`, centralizado na altura | `Jogador.desenhar()`, que usa `Sprites.desenhar()` com o sprite do canhão |
| Escudos | `_criarBarreiras()` chama `Barreira.criarEscudo()` 4 vezes; o formato vem de um desenho em texto (`#` = bloco), girado 90° por `Sprites.girar()` | `Barreira.desenhar()`: um `fillRect` por bloco, mais escuro conforme perde resistência |
| Horda | `_criarOnda()`: lê `Jogo.HORDAS` e cria cada inimigo com `new Tipo(x, y)` (atirador, rápido ou básico) | `Inimigo.desenhar()`: sprite da subclasse (lula, caranguejo ou polvo), alternando 2 quadros para dar efeito de movimento |

O desenho de cada quadro acontece em `Jogo._desenhar()`, chamada pelo `_loop()` 60 vezes por segundo:

```
ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);  // limpa o quadro
this.barreiras.forEach((b) => b.desenhar(ctx));              // escudos
this.jogador.desenhar(ctx);                                  // nave
this.inimigos.forEach((i) => i.desenhar(ctx));               // horda
this.projeteisJogador.forEach((p) => p.desenhar(ctx));       // tiros
this.projeteisInimigo.forEach((p) => p.desenhar(ctx));
```

A ordem das linhas importa, pois o que é desenhado depois fica por cima. O `Jogo` chama `desenhar(ctx)` sem saber o tipo exato de cada objeto e cada classe responde do seu jeito, o que é polimorfismo.

# 7. Testes realizados

Como o ambiente não tinha navegador, a lógica foi testada com Node.js e um DOM simulado, executando as classes reais do projeto. Também foi verificada a sintaxe de todos os arquivos `.js`.

| Teste | Resultado |
|---|---|
| Tiro do jogador contra um bloco do escudo | Resistência continuou em 3 e nenhum bloco foi removido. |
| Tiro inimigo contra um bloco do escudo | Resistência caiu de 3 para 2 (comportamento mantido). |
| `receberDano()` seguido de 52 frames | Vidas passaram de 3 para 2. As cores seguiram a sequência vermelho, verde, vermelho, verde e assim por diante (6 frames cada), terminando em verde. `tempoDano` voltou a 0 e `cor` continuou `#7CFC00`. |
| Tiro inimigo atingindo o jogador pelo `Jogo` | Vida reduzida e efeito de piscar ativado. |
| Posição dos escudos ao iniciar | Escudos entre x = 210 e 258; a nave (até x = 70) nasce sem encostar neles. |

Recomenda-se uma partida no navegador para conferir o resultado visual.

# 8. Conceitos de POO aplicados

- **Encapsulamento:** o estado do efeito ( `tempoDano` ) e a regra de dano ficam dentro de `Jogador`; o `Jogo` não manipula esses detalhes.
- **Responsabilidade única:** o `Jogo` detecta colisões; o `Jogador` sabe reagir ao dano.
- **Sobrescrita de métodos (polimorfismo):** `desenhar()` e `atualizar()` de `Jogador` sobrescrevem os métodos de `Entidade`, agora com o comportamento do efeito de dano.
- **Membros estáticos:** constantes pertencentes à classe, e não a cada objeto.

# 9. Organização dos arquivos

O projeto atualizado está em `space-invaders-final_dano-escudo.zip`. Os arquivos originais, anteriores a estas alterações, estão em `antes/ajuste_dano_escudo/` ( `Jogador_antes.js` e `Jogo_antes.js` ), seguindo o padrão das demais pastas `antes/` do

projeto.